Offloading Online MapReduce tasks with Stateful Programmable Data Planes

Valerio Bruschi, Marco Faltelli, Angelo Tulumello, Salvatore Pontarelli, Francesco Quaglia, Giuseppe Bianchi
CNIT/University of Rome Tor Vergata

***Abstract*—In-network computation based on programmable data plane hardware provides a tremendous opportunity to improve throughput, latency and reduce congestion in data center scenarios. However, a judicious use of these network devices must be done based on their limited resources and the specific features of the application to be offloaded. This paper promotes FlowBlaze, a stateful hardware programmable data plane, as a candidate for offloading online MapReduce tasks. Above all, tasks with strict time requirements can benefit from in-network computing since it can significantly lower their latency. Given that MapReduce is a generic programming paradigm, in this paper we first try to identify which subset of MapReduce operations can be transparently offloaded to a specific hardware architecture and which are the limitations of this offloading in terms of memory and computational resources. After, we show how the FlowBlaze architecture can match the partition/aggregation paradigm and we discuss a set of primitives exposed by the FlowBlaze abstraction to perform mapping and aggregation. Finally, we prove the feasibility of this approach applying it to a click-stream analysis use case.**

I. Introduction

The increasing need for accelerating disparate cloud computing tasks, from transport level ones to L7 applications, gained a significant interest in both research and industry communities in the last years [1]. Indeed, the rise of programmable network devices permits to offload some tasks to specialized hardware and promises several gains in terms of performances. We foresee that in the next future more and more programmable dataplanes will be available in the network, deployed both at the end-nodes (as SmartNICs) and in the core (as programmable switches). This situation will provide a tremendous occasion to opportunistically co-design applications with modern hardware: the application logic could be distributed along multiple accelerators and network devices [2], so that these devices can substitute general purpose CPUs in an offloading-amenable subset of tasks, saving several CPU cycles that can be devoted to keep the most complex tasks in general purpose architectures. Of course, when offloading applications to programmable hardware, the problem of finding the right abstractions to obtain an efficient hardware execution of a meaningful application rises up.

A well known abstraction for big data processing is MapReduce: this paradigm partitions the input data in the *Map* phase and then performs parallel computation and aggregation in the subsequent *Reduce* phase, as explained in more details in Section II. On the one hand, MapReduce has some key limitations: the paradigm struggles with jobs which are not trivially partitionable, stream processing tasks with strict time requirements and most of all, natively sequential tasks, e.g. gradient descent or expectation maximization in ML [3] or a Finite State Machine execution [4]. On the other hand, MapReduce is expressive enough to implement NP hard problems [5], inherently parallel machine learning tasks and, most of all, partition/aggregation applications.

In this paper, we focus on this paradigm and we propose an efficient execution for stream processing tasks on programmable network devices. Stream processing requires computations which are continuously executing over evolving input streams and terminate only upon user request. This has always been a sore point for MapReduce implementation since its programming model is natively suited for batch processing. Prior work has been done in this field with [2] being, to the best of our knowledge, the major contribution: the authors of [2] discuss the theoretical boundaries for in-network computation. With respect to this work, we try to make a step further in this direction through the execution of stream processing tasks on a hardware programmable device with stateful per-flow capabilities, which allows us to perform a broader range of *Reduce* functions. Other works that leverage hardware acceleration propose to offload the entire MapReduce application through an FPGA execution (e.g. [6]). Nevertheless, this approach shows several limitations due to a lack of flexibility and reusability, contrarily our approach follows the idea *"to offload primitives and not entire applications"* [7]. While alternative software approaches addressing the stream processing problem have been proposed, such as MapUpdate [8] or novel frameworks, such as Storm [9] or Flink [10], they reach, at their best, a few milliseconds of latency [11], consequently they still cannot fully meet the strict requirements of many time-sensitive applications [12]. As a example, in High Frequency Trading (HFT), gaining a few microseconds in processing can put a player ahead of the game [13]. We believe that our hardware implementation can: *(i) significantly lessen latency for such tasks (i.e. few nanoseconds for the very same task), (ii) lower the latency variability and (iii) help reducing network traffic, so as to alleviate congestion.*

Our claim is not to offload *all* of the stream processing tasks to dedicated hardware, but rather to identify a specific subset of them with amenable features for high-performance execution on a stateful programmable networking device: our first result is to have found similarities between the MapReduce paradigm and the internal architecture of FlowBlaze, as we

978-1-7281-5127-4/20/$31.00 ©2020 IEEE

discuss in Sections III-A, III-B. In Section III-C, we discuss how the FlowBlaze nodes can be placed in the network for this scenario. We identify in Sections III-D, III-E a set of primitives for both the *Map* and the *Reduce* phases, which are amenable for a hardware implementation. Finally, in order to gather insights on the gain achievable with our approach, we experiment it in a specific click-stream analysis use case over HTTP traffic, as shown in Section IV. Conclusions are drawn in Section V.

II. MapReduce

MapReduce is a programming paradigm for processing large data sets in distributed environments [14]. A standard MapReduce process handles terabytes or even petabytes of data on interconnected systems forming a cluster of nodes. MapReduce is composed of two phases: a *Map* phase and a *Reduce* phase. The *Map* phase processes a generic input to generate a set of intermediate $\langle key, value \rangle$ pairs. The *Reduce* phase merges all the intermediate values associated with the same key. *Map* invocations are distributed through multiple instances, each receiving as input a split of the incoming data. Reductions are distributed too, via the partition of the intermediate key space into R pieces through a split function.

The *"hello world"* example of a MapReduce application is the word count one, which counts the occurrences of each word in a set of documents. The *Map* phase is responsible for splitting documents into words and producing a $\langle key, value \rangle$ pair for every word, while the reduce phase groups by word and sums the occurrences of the received values. Figure 1 shows a graphical representation for this application.

Another example is a reverse web-link graph: the *Map* function gets as input a *source* web page and emits a $\langle target, source \rangle$ pair for each target URL found in this page. The reduce function concatenates the source URLs within the same target, producing a $\langle target, list(source) \rangle$ output.

A more sophisticated example implements a log processing task used by web service owners in order to track and analyze customer behaviours: providers may execute a click-stream analysis on multiple user sessions and compute three different metrics:

1) the number of user sessions, which can be reduced to the previous word count example, where a single counter is incremented for every new user session;
2) the average number of clicks per session, the *Map* phase splits the different session IDs and the *Reduce* phase counts the *HTTP.GET* messages for each session;
3) the average session duration, where the *Map* phase splits the different session IDs and the *Reduce* phase computes the difference between the timestamps of the last and the first session messages.

A. Executing MapReduce on programmable hardware

The MapReduce programming paradigm is quite generic and can be applied to a wide set of scenarios. Therefore we believe that it is not feasible to devise a full and transparent offload of the MapReduce functionalities while retaining the

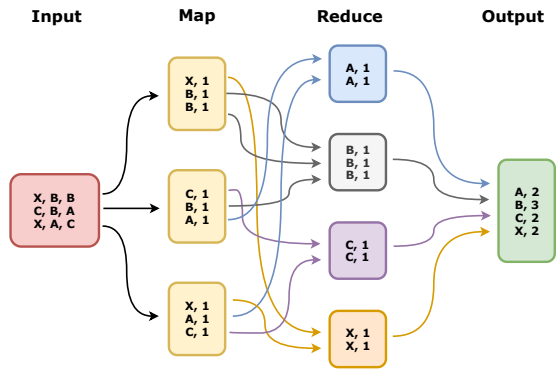

Fig. 1. The MapReduce paradigm applied to the WordCount example: the input file is split and processed by the mappers which produce letters as different keys. Each reducer counts the occurrences for the same letter and an output with the number of occurrences for each letter is produced.

possibility of gains in its efficiency. Instead, we believe that it is possible to identify a subset of the MapReduce operation that can be easily moved to specific hardware acceleration platforms. We also believe that the specific subset is dependent on the application scenario and on the specific hardware accelerator architecture. For example, a generic FPGA is more flexible than a programmable data plane, but the programming effort to use a generic FPGA is much higher. Furthermore, a programmable data plane is naturally related to the offloading of network related computation ($e.g.$ the above mentioned click-stream analysis) and is also located in the right position to perform some *Reduce* tasks [2]. To identify the set of to-be-offloaded operations that are suited for a programmable data plane, we first underline that the *Map* phase is too generic, since it considers any possible information type as the input to produce the intermediate $\langle key, value \rangle$ pairs. On the other hand, a programmable data plane takes as input a structured packet formed by a set of headers. Therefore, we believe that it is necessary to restrict the possible inputs of the *Map* phase to a well defined structure composed by a fixed number of $\langle key, value \rangle$ pairs, which can be easily extracted by a programmable parser [15].

This is the same idea followed by other approaches dealing with hardware acceleration of MapReduce Frameworks. For example, in [16], the generic information in input to the *Map* phase is pre-processed by a CPU, encapsulated as messages and sent using a PCIe interface to an FPGA which accelerates the actual map computation. The same approach is proposed in [2], where each packet delivers a fixed number of $\langle key, value \rangle$ pairs, which are extracted by the P4 parser and used as input for the map and reduce tasks. It is worth to notice that for many networking related computations, which already work on a packet basis, this pre-processing task can be avoided, since the data to be processed are already available as header fields in the packet. As an example, the click-stream analysis on HTTP traffic can be done with no pre-processing, if the programmable parser is able to process up to the L7 layer of the HTTP request.

After the definition of the input type, we also highlight

that the actual offloading capability depends on the specific computational task required by the *Map* phase. The set of supported operations in a programmable data plane is rather small and limited to arithmetic, hashing and data manipulation primitives [2]. Moreover, these network devices must still perform at line rate, so the computation that can be done for every packet is highly restricted and very few operations of the above mentioned type can be performed. However, even if this seems a huge restriction there are multiple use cases (as those listed in Section II) where they seem sufficient for offloading the *Map* phase. Furthermore, these primitives meet the standards for most of the aggregation functions, e.g. average, minimum, sum, etc. Such functions are great candidates for a hardware execution since they can be fastly executed and easily parallelized. They also benefit from the cumulative and associative properties, thus permitting partial aggregations without altering the final result. Therefore the offloading of the *Reduce* phase seems not to be limited by the type of supported aggregation function. Instead, differently from the *Map* phase, the *Reduce* phase is more memory demanding, since it requires to accumulate intermediate results for each key value until the computation of the final result. As it will be discussed in more details in Section III-D, programmable data planes have a limited internal memory and the scalability of the *Reduce* phase can be limited by the required amount of memory depending on the application.

Finally, we highlight that, from the point of view of the programmable data plane, the *Map* phase seems mainly stateless, given that it is composed by a parsing operation followed by a set of arithmetic/logic operations which directly produce the $\langle key, value \rangle$ pair for the reducers. Both these operations do not leave states into the data plane, so they can be easily supported by stateless data planes. Instead, the *Reduce* phase seems inherently stateful, since it stores for each key a set of intermediate values which are used to accumulate the partial results[1].

III. FlowBlaze as high performance MapReduce engine

FlowBlaze [17] is a stateful programmable data plane that was originally developed as an accelerator for network functions able to exploit SmartNIC benefits [18]. FlowBlaze is a pipeline of programmable stages and each stage can be configured to provide a stateless functionality similar to a Match-Action Table or a stateful per-flow functionality defined by means of a XFSM (eXtended Finite State Machine) [19]. As mentioned before, in this paper we investigate the opportunity to offload "partition/aggregation" applications to a FlowBlaze engine. In this section we present the FlowBlaze architecture, its use as a mapper-reducer executor and we discuss the use

[1]Even if a PISA architecture can use registers to store intermediate values, this is more a workaround than an actual stateful operation. In fact, an algorithm like the one proposed in [2] has several limitations: it relies on a spillover to resolve hash collisions and has a severe memory limitation since the register array has a memory size much smaller than flow tables such as those used by a natively stateful data plane such as FlowBlaze.

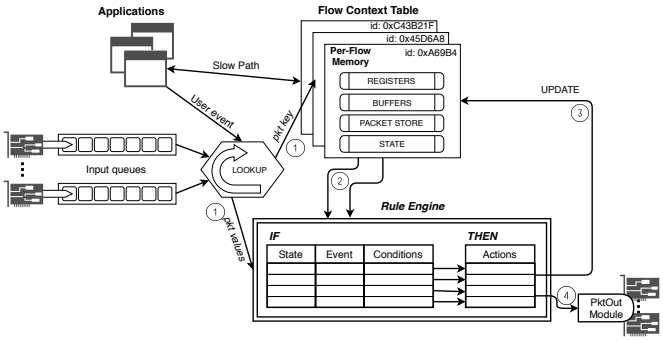

Fig. 2. Sketch of the FlowBlaze abstraction. The **lookup** identifies the context depending on the key. The actions and the state transitions are selected by the **Rule Engine**. After, the results are updated in the **Flow Context Table** and the actions for the packet are executed.

of a specific high level language for designing map reduce based applications that can be executed by FlowBlaze. Later, we identify the network locations where FlowBlaze can be deployed and we present some of the primitives exposed by FlowBlaze for mapping and aggregation.

A. MapReduce phases on FlowBlaze

Our first result is to have found similarities between the MapReduce paradigm and the internal architecture of FlowBlaze. We explain this aspect in few steps, looking at the internal architecture of a FlowBlaze stage as depicted in Figure 2:

1) FlowBlaze produces a $\langle key, value \rangle$ pair from each incoming packet by parsing the packet.
2) The key is used to identify the corresponding context for that packet. This means that FlowBlaze retrieves the partial results stored in the context table.
3) The values and the partial results loaded from the context table are used by FlowBlaze to perform reduction. The results are then updated in the context table.
4) If needed, FlowBlaze can generate packets which act as triggers.

In the MapReduce paradigm, the *Map* phase produces a set of intermediates key-value pairs from the input, as FlowBlaze does from the packet (①-②). The *Reduce* phase aggregates all the intermediate values, as FlowBlaze does for each packet within the same flow (②-③-④). It is worth noticing that compared to the original paradigm, FlowBlaze allows (i) implementing both the map and the reduce phases on the same device and (ii) managing different keys at the same time without breaking up the reduce phase.

B. XL code for MapReduce

As stated before, the FlowBlaze architecture natively supports the MapReduce paradigm due to its ability to process XFSMs. We believe that XFSMs, as formally specified in [20], appear to well fit a wide range of applications, since they permit to transform the problem of abstracting a desired function into a list of structured "if-then" statements, as described in Figure 3. In the *if* part, there are three types of information: the current state label, the event which triggers the rule (e.g.

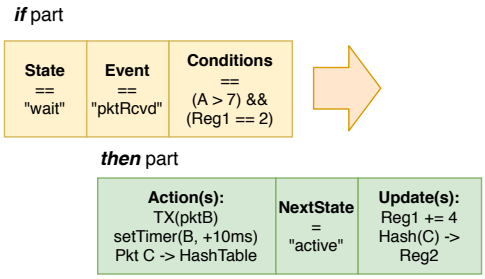

Fig. 3. Example of an XFSM rule entry

a packet is received), and the conditions that must be verified to make the transition ($A > 7$ && $Reg1 == 2$). The *if* part is the natural implementation of a *Map* phase: from one side the *if* part identifies the packet context and, on the other side, selects the actions and updates to perform. Next, the effects of the state transition are described in the *then* part, where we find the action list to be executed (e.g. perform reduction, drop the packet, etc), the XFSM next state and the update instructions for the per-flow registers (i.e. the partial aggregation results). Therefore, the *then* part is the natural actor of a *Reduce* phase: it allows to perform operations between the information carried by the packet and per-flow registers where partial aggregation results are stored.

For the above mentioned reasons we propose to describe MapReduce jobs using the XL language [21], which has been expressly developed to describe a sequence of XFSMs to be executed by a programmable stateful data plane. The use of an high level language such as XL also enables the deployment of MapReduce jobs in an automatic and dynamic way.

C. Network Placement

So far, we have discussed the similarities between the *Map* and *Reduce* phases and FlowBlaze. However, the aim of the paper is not to replace the MapReduce paradigm, but to offload Map and Reduce tasks to programmable data planes in order to achieve lower latency processing. We notice that one of the main aspects of the MapReduce paradigm is scalability: this is usually achieved through highly parallelized and distributed execution over a large number of nodes, and we do not want to alter this architecture. Nevertheless, we propose to assign Map and Reduce tasks in a distributed way to network data planes (as FlowBlaze can do) and to further exploit the opportunity that these devices offer to execute both Map and Reduce jobs in the same device. In particular, depending on the specific MapReduce task and on the network architecture it is possible to deploy (or to use) FlowBlaze nodes placed in different network locations to accelerate this kind of tasks. In a classical fat tree data center topology a FlowBlaze node can be placed as an intermediate network agent that performs both the standard network operations and also a MapReduce task. However, if a limited number of hardware devices are available in the data center topology, our approach needs a routing algorithm in order to forward the targeted traffic to the FlowBlaze instances. Furthermore, it could be also used as a SmartNIC on a single point to perform the final reduction task, reducing CPU utilization. For example, in the application taken into account in Section IV, FlowBlaze is deployed as an intermediate node that forwards HTTP traffic and performs the click stream analysis. The optimal placement of FlowBlaze nodes is obviously dependent on the overall scenario, since it is necessary to take into account both network related constraints and the characteristics of the MapReduce applications to offload.

D. FlowBlaze primitives for mapper

As mentioned before, the *Map* phase consists mainly in extracting the $\langle key, value \rangle$ pair from the incoming packets. This extraction could require to parse the protocol stack of the packet up to L7. For example, in the click stream analysis described in Section IV the engine must extract the key parsing the standard 5-tuple and also the HTTP request type. Generally speaking, the applications targeted by MapReduce usually transport data inside UDP packets, thus requiring ad-hoc parsing of the incoming packets. This flexible protocol parsing is a standard feature of programmable data planes and is supported by languages such as P4 [22]. Even if hardware implementations of programmable parsers have been presented in the literature [23], [24] it is worth to notice that the parsing of upper protocols is complex and can generate bottlenecks in the pipeline and reduce the applicability of this devices. A possible solution to solve this bottleneck is the parallelization of parsing, which alleviates the throughput limitations or the use of standardized message interfaces that reduce the packet parsing complexity.

A second typical aspect of MapReduce operations is that they are applied concurrently to thousand of tasks in parallel. The per-flow capabilities of the FlowBlaze approach seamlessly provide this support to concurrency with a negligible context switching overhead. The number of concurrent tasks supported by the FlowBlaze engine depends on the number of registers required by the specific application and by the overall amount of available memory. We developed 2 flavours of FlowBlaze, one based on DPDK,other based on FPGA. In the software implementation, we expect that increasing the memory usage the overall performances will decrease, as it will be shown in Section IV. Instead, for the FPGA implementation the memory is bounded by the FPGA internal memory. While the proof-of-concept presented in [17] has a quite limited amount of internal memory, it is worth to notice that last generation FPGA provides up to 500Mb of total on-chip integrated memory, plus up to 16GB of high-bandwidth memory (HBM) Gen2 integrated in-package for 460GB/s of memory bandwidth.

E. FlowBlaze primitives for aggregation

As a primary rule, in network applications deployed on line rate switches should not worsen the device performances: therefore, the computation that can be done on each packet is restricted to very few clock cycles. In order to meet such a constraint, FlowBlaze exposes a finite set of primitives which allow to perform incremental computation on the information

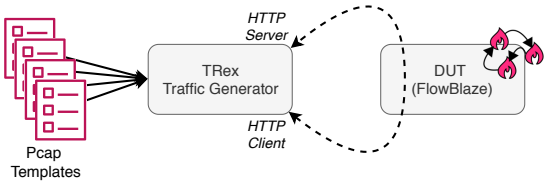

Fig. 4. Testbed architecture: TRex gets a series of traffic templates as input and reproduces them establishing connections between multiple clients and servers; FlowBlaze is placed in between the client and server interfaces, snooping all the traffic between them and computing the targeted metrics.

carried out by each packet. Nevertheless, these primitives are sufficient to implement the typical aggregation functions, which satisfy both the *associative* and *commutative* properties [25], namely sum, mean, max, min, etc. This constraint can actually become an opportunity to meet the strict time requirements in some real time application use cases, e.g. financial trading or smart grid price modeling, etc.

IV. Experimental Results

A. Use case description

In order to gather some preliminary results, we developed a simple click-stream analysis over HTTP traffic from the literature [26] to validate our proposed approach. The MapReduce task should compute three different metrics through the overall HTTP traffic:

1) the number of user sessions (*group by 5-tuple and count*)
2) the average number of clicks per session (*for each session group by HTTP.GET and count*)
3) the average session duration (*for each session group by server.id and average session.time*)

It is worth noticing that, even if nowadays most of the traffic is encrypted using an HTTPS session, the above metrics can still be computed since in many cases the servers actually providing the response to the GET request are behind a TLS termination proxy [27].

B. Testbed description

Figure 4 sketches the key components used in our experimental campaign: namely, the Cisco TRex traffic generator [28] and the FlowBlaze Device Under Test (DUT).

TRex is a free and open source software based on DPDK [29] for fast (up to 200 Gbit/s) network traffic generation maintaining a very precise timing. Moreover, it leads to configure and generate, in a flexible and accurate way, application layer payloads. The traffic generated by TRex is defined by multiple traffic templates: a template may include multiple TCP flow samples in the form of pcap files. Templates allow to shape the traffic properties (like flow connections per second, inter packet timing, etc.) according to the application scenario. In our case, we tuned these parameters in order to saturate a 10 Gb link, using a template with a fixed number of 20 HTTP GET requests per session and a session duration of 140 ms. While these parameters were kept fixed during the tests, we doubled the number of total flows for each test, starting from 4k to 512k. This choice allowed us to capture the effects of workload scaling on the application performance, as it will be shown in Section IV-C.

As for the DUT FlowBlaze architecture, we used the software implementation which is available on GitHub [30]. Our software implementation is also based on DPDK, providing a high-performance framework able to handle the traffic sent from TRex at 10 Gbps with zero packet loss. We remark that, even if we developed both a DPDK based implementation and a FPGA based one of FlowBlaze [17], for the test we focused only on the software implementation. The difference between the software and the hardware implementations is mainly related to the scaling of the application with the number of flows. The software implementation leads to an increase of the processing latency with the increase of the number of flows to process. Instead, the FPGA implementation processes flows at line rate with a fixed latency which is independent of the number of flows [17]. As mentioned in Section III-D, the hardware limitation is in the maximum number of supported flows, which is currently 64K. This limit is caused by the old FPGA device hosted by the NetFPGA SUME board used to prototype FlowBlaze and can be easily superseded using a last generation FPGA. Thus, since the current prototype would give few insights on the behaviour of the MapReduce application with respect to session scaling, we focused only on the software implementation.

C. Results

Tests were conducted on a 12 physical core server (running Intel Xeon Silver @2.10 GHz without the Intel TurboBoost feature) with 128 GB RAM memory. Figure 5 shows the results from our test: in a first scenario (highlighted with triangular dots), they revealed that FlowBlaze was losing no packet. Consequently, there were no losses or re-transmissions, so FlowBlaze correctly measured 20 HTTP GET requests for session and 140 ms as the average session duration, independently of the number of total flows. In order to find a more challenging scenario, a second test (highlighted with squared dots) was conducted: in this case, we downgraded the CPU clock speed to 1.8 GHz. As shown by the plot, between 32k and 64k flows FlowBlaze started losing packets, so an HTTP GET re-transmission may have occurred, increasing the average number of GET instances per session. The average session duration also increased, because of both longer queue delays and packet losses. The number of total user sessions was correctly calculated by FlowBlaze in both cases.

V. Conclusions and Future Work

In this work, we have tackled the issue of high throughput and low-latency processing for stream based analysis, by exploring the capability that hardware acceleration can offer. Indeed, in-network computing is a strong trend that promises to reduce latency and consumption of server resources by offloading tasks to network programmable data planes. Following this direction, we have proposed to offload "partition/aggregation" applications inside a stateful programmable data plane called FlowBlaze. In particular we focused on the

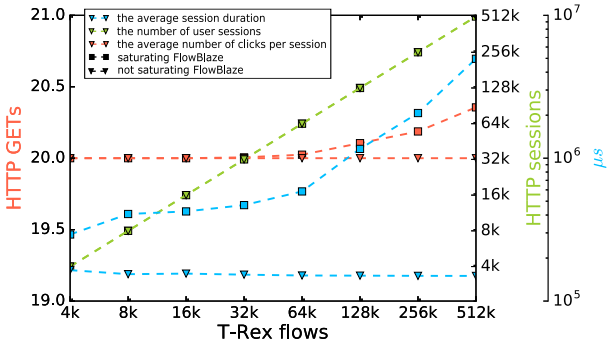

Fig. 5. Click stream analysis results computed by the FlowBlaze engine.

well known MapReduce paradigm and we discussed how the FlowBlaze abstraction natively supports its tasks. In particular, the per-flow stateful operations provided by FlowBlaze lead to manage multiple concurrent tasks and to easily aggregate partial results in the internal memory, allowing the offloading of the MapReduce tasks. Moreover, we have shown that FlowBlaze allows (i) implementing both the *Map* and *Reduce* phases on the same device and (ii) managing different keys at the same time without breaking up the reduce phase.

Finally, we assessed our proposal with a use case related to a simple click-stream analysis over HTTP traffic, demonstrating the feasibility of our approach. In particular, the FlowBlaze DPDK implementation running on a single core clocked at 2.10 GHz is able to correctly perform the offloaded task, even at the throughput of a 10 GbE interface saturated link and up to 512K different flows. To better understand the limitations of our engine, we decreased the CPU frequency so as to generate an impact on the end to end application. This allows us to identify when it is needed to allocate more cores to the FlowBlaze engine to sustain higher throughput.

While, the collected results provide some insights on the gain achievable with our approach, we also point to future steps that can improve our vision:

1) Integrating the XL toolchain in a standard MapReduce framework can allow to directly configure FlowBlaze in a distributed environment.
2) Validating our approach in a wider range of "partition/aggregation" applications which require heavier reduction operations on the input data without altering our assumptions.
3) Exploiting a standardized message interface to transmit data to FlowBlaze leads to improve the usability of our approach. The use of these messages can avoid the limitation (as mentioned in Section III-D) about the data extraction by complex packet protocol parsing.
4) The per-flow abstraction can natively support the concurrent execution of multiple diverse MapReduce tasks. By enabling this feature we will permit to expose FlowBlaze as a multi-tenancy Function-as-a-Service (FaaS) engine.
5) Comparing FlowBlaze and P4, through the P4 $\rightarrow$ NetFPGA workflow [31], in order to extend our experimental campaign by using the same FPGA device.

Acknowledgment

This work is partially supported by the EU Commission in the frame of the H2020 project 5G-PICTURE (grant #762057).

References

[1] A. Caulfield, P. Costa, and M. Ghobadi, "Beyond SmartNICs: Towards a fully programmable cloud," in *IEEE HPSR*, 2018.
[2] A. Sapio, I. Abdelaziz, A. Aldilaijan, M. Canini, and P. Kalnis, "In-Network Computation is a Dumb Idea Whose Time Has Come," in *ACM SIGCOMM HotNets Workshop*, 2017.
[3] K. Grolinger, M. Hayes, W. A. Higashino, A. L'Heureux, D. S. Allison, and M. A. M. Capretz, "Challenges for MapReduce in Big Data," in *2014 IEEE World Congress on Services*, 2014.
[4] K. Asanovic, R. Bodik *et al.*, "The Landscape of Parallel Computing Research: A View from Berkeley," UC BERKELEY, Tech. Rep., 2006.
[5] I. Bezáková, "NP-Hard problems using Map-Reduce," Ph.D. dissertation, Rochester Institute of Technology, 2013.
[6] T. Honjo and K. Oikawa, "Hardware acceleration of Hadoop MapReduce," in *2013 IEEE International Conference on Big Data*, Oct 2013.
[7] D. R. Ports and J. Nelson, "When Should The Network Be The Computer?" in *HotOS 2019*. ACM, 2019.
[8] W. Lam, L. Liu, S. Prasad, Rajaraman *et al.*, "Muppet: MapReduce-style processing of fast data," *Proceedings of the VLDB Endowment*, 2012.
[9] Apache Storm. [Online]. Available: https://storm.apache.org/
[10] Apache Flink. [Online]. Available: https://flink.apache.org/
[11] T. Kostas, "High-throughput, low-latency, and exactly-once stream processing with apache flink," 2015.
[12] A. Osman, M. El-Refaey, and A. Elnaggar, "Towards Real-Time Analytics in the Cloud," in *IEEE Ninth World Congress on Services*, 2013.
[13] Arnuk, S. and Saluzzi, J., "Latency arbitrage: The real power behind predatory high frequency trading," *Themis Trading white paper*, 2009.
[14] J. Dean and S. Ghemawat, "Mapreduce: simplified data processing on large clusters," *Communications of the ACM*, 2008.
[15] G. Gibb, G. Varghese, M. Horowitz, and N. McKeown, "Design principles for packet parsers," in *IEEE ANCS*, 2013.
[16] Y. Shan *et al.*, "FPMR: MapReduce framework on FPGA," in *2010 ACM/SIGDA Int. Symposium on FPGA*.
[17] S. Pontarelli, R. Bifulco, M. Bonola, C. Cascone, M. Spaziani, V. Bruschi, D. Sanvito, G. Siracusano, A. Capone, M. Honda, F. Huici, G. Siracusano, and G. Bianchi, "FlowBlaze: Stateful Packet Processing in Hardware," in *NSDI'19*.
[18] D. Firestone *et al.*, "Azure Accelerated Networking: SmartNICs in the Public Cloud," in *NSDI'18*.
[19] V. S. Alagar and K. Periyasamy, *Extended Finite State Machine*. Springer London, 2011.
[20] K. T. Cheng and A. S. Krishnakumar, "Automatic Functional Test Generation Using The Extended Finite State Machine Model," in *ACM Int. Design Automation Conference (DAC)*, 1993.
[21] G. Bianchi, M. Welzl, A. Tulumello, F. Gringoli, G. Belocchi, M. Faltelli, and S. Pontarelli, "XTRA: Towards Portable Transport Layer Functions," *IEEE TNSM*, 2019.
[22] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM CCR*, 2014.
[23] G. Gibb, G. Varghese, M. Horowitz, and N. McKeown, "Design principles for packet parsers," in *Architectures for Networking and Communications Systems*. IEEE, 2013.
[24] J. Cabal, P. Benáček, L. Kekely, M. Kekely *et al.*, "Configurable FPGA packet parser for terabit networks with guaranteed wire-speed throughput," in *2018 ACM/SIGDA International Symposium on FPGA*.
[25] Y. Yu, P. K. Gunda, and M. Isard, "Distributed Aggregation for Data-parallel Computing: Interfaces and Implementations," in *ACM SOSP'09*.
[26] D. Logothetis, C. Trezzo, K. C. Webb, and K. Yocum, "In-situ MapReduce for Log Processing," ser. USENIX ATC'11, 2011.
[27] M. O'Neill, S. Ruoti *et al.*, "TLS proxies: Friend or foe?" in *Proceedings of the 2016 Internet Measurement Conference*. ACM, 2016.
[28] [Online]. Available: trex-tgn.cisco.com
[29] Intel. Data Plane Development Kit. [Online]. Available: https://dpdk.org
[30] FlowBlaze. [Online]. Available: https://github.com/axbryd/FlowBlaze
[31] S. Ibanez, G. Brebner, N. McKeown, and N. Zilberman, "The P4 $\rightarrow$ NetFPGA Workflow for Line-Rate Packet Processing," in *2019 ACM/SIGDA International Symposium on FPGA*.